Sistema de Gestión de Productos

Proyecto del Módulo 4 · Escuela Musk · 31 octubre 2026

Un inventario. Dos tipos de producto. Una API para gestionarlos.

Java 17 · Maven · Spring Data JPA · H2

Qué resuelve la aplicación

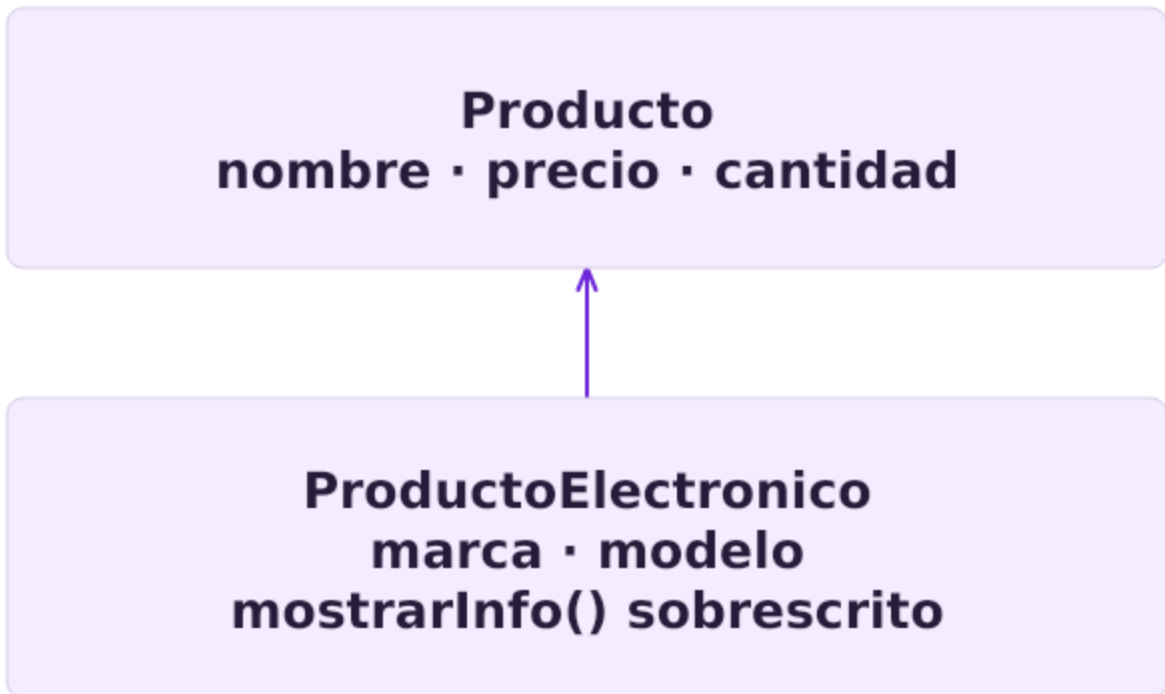
El cliente REST envía datos y recibe respuestas JSON.

CREAR Y CONSULTAR	Alta de productos, listado y consulta por identificador.
ACTUALIZAR Y BORRAR	Cambiar datos y retirar productos del inventario.
BUSCAR Y COMPARAR	Filtrar por nombre y encontrar el mayor stock.

H2 en memoria: los datos se reinician al detener la aplicación.

POO aplicada al modelo

Los campos privados se modifican mediante métodos que validan los datos.



Implementa un contrato

Herencia y comportamiento específico de la subclase

Del array al inventario dinámico

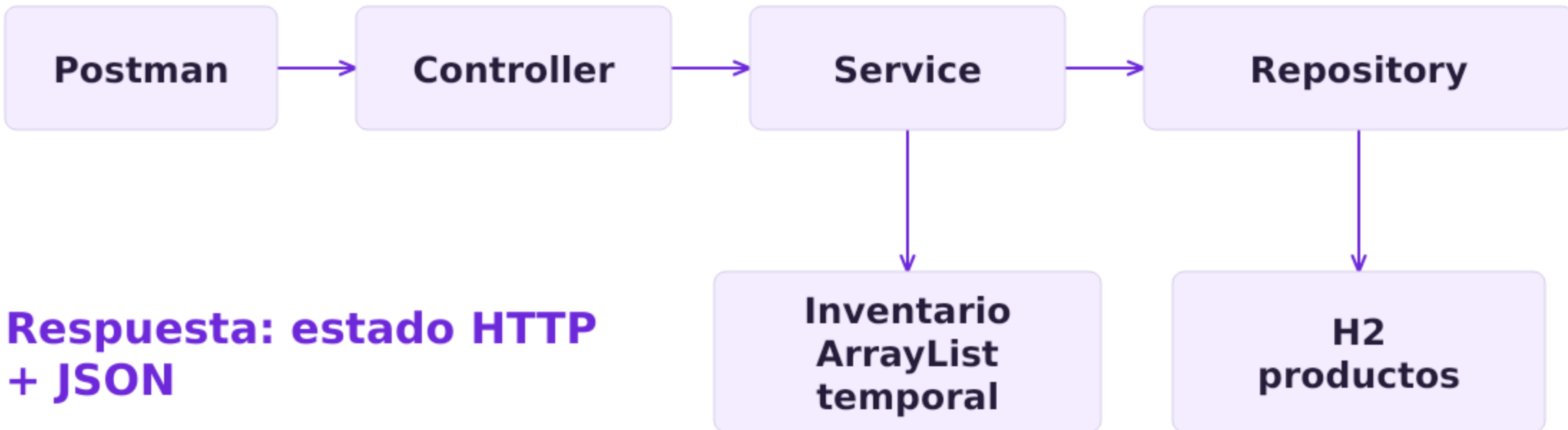
Inventario concentra las operaciones que recorren los productos.

ARRAY	Producto[] contiene los ejemplos iniciales de la demo.
ARRAYLIST	Inventario.agregar() incorpora cada producto.
BUCLES	mostrarTodos(), buscarPorNombre() y mayorCantidad().

En la API, H2 es la fuente de datos; la lista se reconstruye al consultar.

Cómo viaja una petición

Un solo proceso Spring Boot, organizado por responsabilidades.



Respuesta: estado HTTP + JSON

Un contrato REST sencillo

Base local: http://localhost:8081

GET	/productos · /productos/{id}
POST	/productos → 201 Created
PUT / DELETE	/productos/{id} → 200 / 204
CONSULTAS	?nombre=port · /productos/mayor-stock

Las entradas inválidas se rechazan

HttpException y un manejador central producen errores comprensibles.

400	Datos inválidos: precio negativo, nombre vacío o JSON incorrecto.
404	El identificador es válido, pero el producto no existe.
409	Se intenta convertir un producto existente a otro tipo.

Un PUT rechazado conserva los datos anteriores.

Monolito frente a microservicios

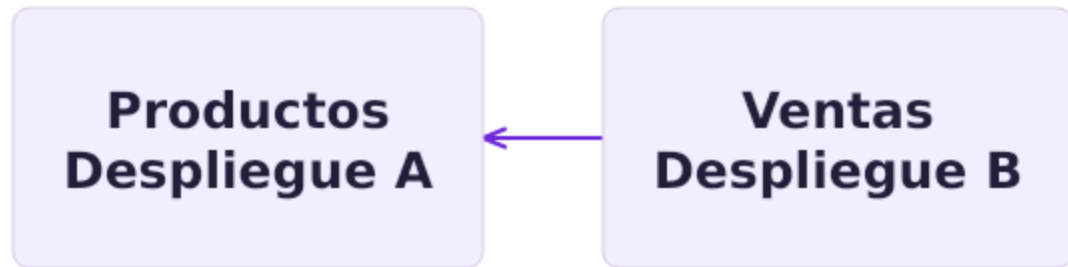
Decisión del proyecto: un monolito por capas.

MONOLITO · IMPLEMENTADO

API + negocio + persistencia
Un despliegue

Ejecución y pruebas simples.
Se publica el conjunto.

MICROSERVICIOS · ALTERNATIVA



Cambios independientes.
Coordinación entre servicios.

MVC y arquitectura hexagonal

Describen responsabilidades internas; pueden combinarse.

SPRING MVC · EN ESTA API

Controller → Service → Modelo
Respuesta JSON

Organiza la capa web.
Sin vista HTML en esta entrega.

HEXAGONAL · ALTERNATIVA

Adaptadores → Puertos
Casos de uso y dominio

Aísla el núcleo de HTTP y BD.
Requiere separar dependencias.

Patrones que aparecen en el código

Cada separación tiene una responsabilidad concreta.

REPOSITORY	ProductoRepository: acceso a datos.
SERVICE LAYER	ProductoService: reglas y transacciones.
DTO	ProductoRequest / Response: contrato HTTP.
INYECCIÓN	Dependencias proporcionadas por constructor.

Verificación del comportamiento

La compilación ejecuta pruebas automáticas antes de generar el JAR.

16

pruebas automáticas
7 de modelo + 9 de API

DATOS	Persistencia de la subclase y operaciones CRUD.
ERRORES	400, 404, 409 y conservación de datos al rechazar PUT.

Demostración y entrega

Código Java, diagrama comparativo y presentación.

1 · POO	Ejecutar el JAR con --demo.
2 · API	Alta → consulta → actualización → error → eliminación.
3 · ENTREGA	Publicar en GitHub y completar el documento del enlace.

Ampliaciones: autenticación, paginación y persistencia en disco.